

EBD

Last edited by [Daniel Gomes Silva](#) 3 months ago

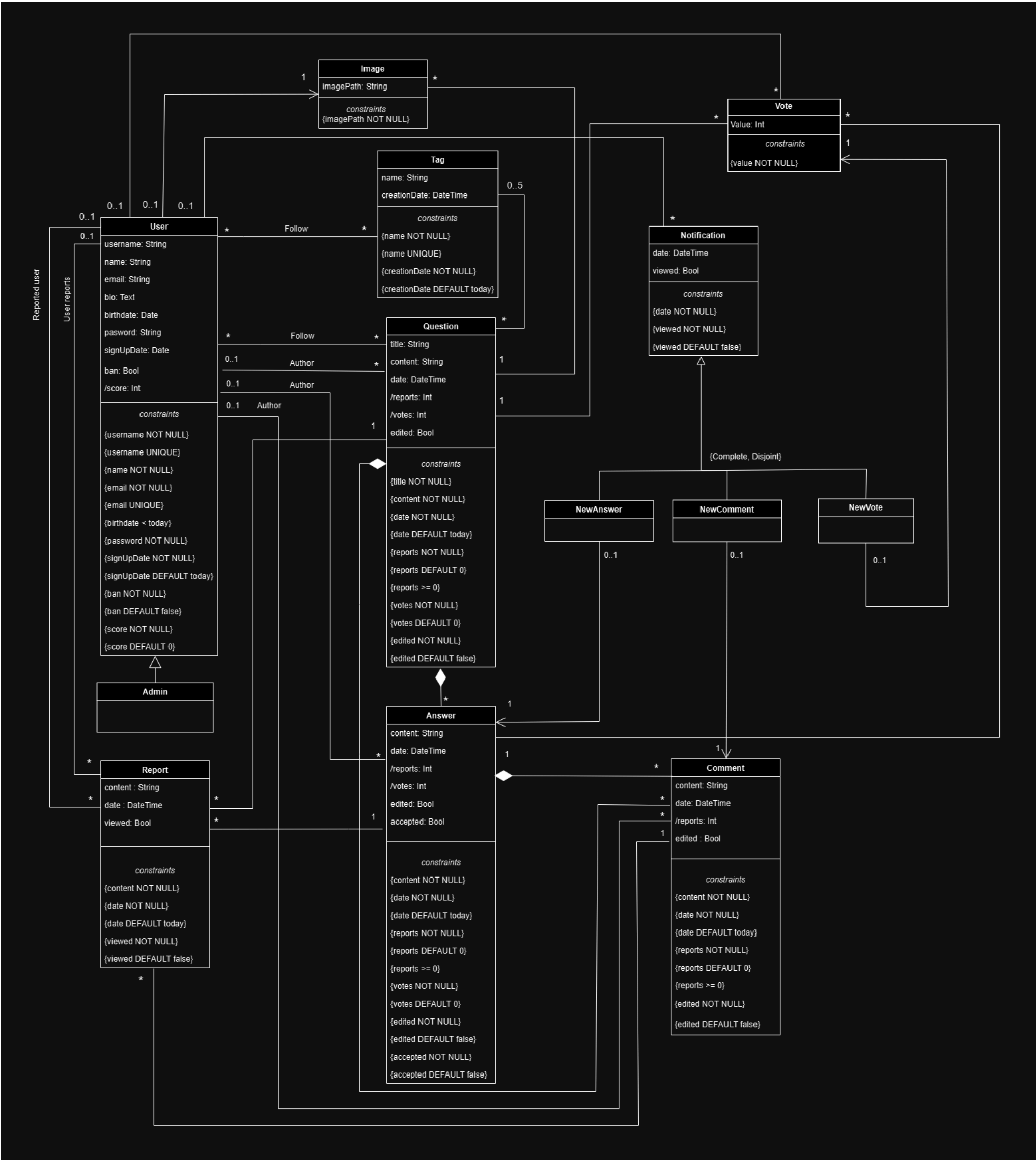
EBD: Database Specification Component

Creating a space where developers can quickly find accurate, reliable answers to their coding questions. What sets it apart is the community-driven approach, where users contribute their expertise, and the platform's voting and reputation system rewards quality contributions. We hope to create a self-sustaining cycle of knowledge.

A4: Conceptual Data Model

The goal of A4 is to identify the entities of the domain and the relationships between them.

1. Class diagram



2. Additional Business Rules

Identifier	Description
BR01	An answer can only be marked accepted by the question author.
BR02	A user cannot report the same answer/comment/question/user more than once.
BR03	A report has to be associated with one and only one of the following: user/answer/comment/question.
BR04	A question can only be associated with a maximum of 5 tags.

A5: Relational Schema, validation and schema refinement

The objective of A5 is to represent the relational schema mapped from the conceptual data model, and also to validate the schema to prove that it is in the requested normal form.

1. Relational Schema

Relation reference	Relation Compact Notation
R01	user (<u>id</u> , admin NN DF false, username UK NN , name NN , email UK NN , bio, birthdate CK birthdate < Today, password NN , signUpDate NN DF Today, ban NN DF false, score NN DF 0)
R02	report (<u>id</u> , content NN , date NN DF Today, viewed NN DF false, id_user → user, id_question → question, id_answer → answer, id_comment → comment) CK ((id_user NN AND id_question IS NULL AND id_answer IS NULL AND id_comment IS NULL) OR (id_user IS NULL AND id_question NN AND id_answer IS NULL AND id_comment IS NULL) OR (id_user IS NULL AND id_question IS NULL AND id_answer NN AND id_comment IS NULL) OR (id_user IS NULL AND id_question IS NULL AND id_answer IS NULL AND id_comment NN))
R03	tag (<u>id</u> , name UK NN , creationDate NN DF Today)
R04	question (<u>id</u> , title NN , content NN , date NN DF Today, reports NN DF 0 CK reports >= 0, votes NN DF 0, edited NN DF false, id_user → user)
R05	answer (<u>id</u> , content NN , date NN DF Today, reports NN DF 0 CK reports >= 0, votes NN DF 0, accepted NN DF false, edited NN DF false, id_user → user, id_question → question)
R06	comment (<u>id</u> , content NN , date NN DF Today, reports NN DF 0 CK reports >= 0, edited NN DF false, id_user → user, id_question → question, id_answer → answer) CK ((id_question NN AND id_answer IS NULL) OR (id_question IS NULL AND id_answer NN))
R07	notification (<u>id</u> , date NN DF Today, viewed NN DF false , id_user → user, id_answer → answer, id_comment → comment, id_question_vote → question_vote, id_answer_vote → answer_vote) CK ((id_answer NN AND id_comment IS NULL AND question_vote_id IS NULL AND answer_vote_id IS NULL) OR (id_answer IS NULL AND id_comment NN AND question_vote_id IS NULL AND answer_vote_id IS NULL) OR (id_answer IS NULL AND id_comment IS NULL AND question_vote_id NN AND answer_vote_id IS NULL) OR (id_answer IS NULL AND id_comment IS NULL AND question_vote_id IS NULL AND answer_vote_id NN))
R08	follow_tag (<u>id_user</u> → user, <u>id_tag</u> → tag)
R09	follow_question (<u>id_user</u> → user, <u>id_question</u> → question)
R10	question_vote (<u>id</u> , id_user → user, id_question → question , value CK (value = 1 OR value = -1))
R11	answer_vote (<u>id</u> ,id_user → user, id_answer → answer, value CK (value = 1 OR value = -1))
R12	image (<u>id</u> , imagePath NN , id_user → user, id_question → question) CK ((id_question NN AND id_user IS NULL) OR (id_question IS NULL AND id_user NN))
R13	question_tag (<u>id_question</u> → question, <u>id_tag</u> → tag)

Relational Model Legend:

- UK = UNIQUE KEY
- NN = NOT NULL
- DF = DEFAULT
- CK = CHECK
- Primary keys are underlined

2. Domains

The specification of additional domains can also be made in a compact form, using the notation:

Domain Name	Domain Specification
Today	DATE DEFAULT CURRENT_DATE

3. Schema validation

TABLE R01	user
Keys	{ id }
Functional Dependencies:	
FD0101	id → {admin, username, name, email, bio, birthdate, password, signUpDate, ban, score}
NORMAL FORM	BCNF

TABLE R02	report
Keys	{ id }
Functional Dependencies:	
FD0201	id → {content, date, viewed, id_user, id_question, id_answer, id_comment}
NORMAL FORM	BCNF

TABLE R03	tag
Keys	{ id }
Functional Dependencies:	
FD0301	id → {name, creationDate}
NORMAL FORM	BCNF

TABLE R04	question
Keys	{ id }
Functional Dependencies:	
FD0401	id → {title, content, date, reports, votes, edited, id_user}
NORMAL FORM	BCNF

TABLE R05	answer
Keys	{ id }
Functional Dependencies:	
FD0501	id → {content, votes, date, reports, accepted, edited, id_user, id_question}
NORMAL FORM	BCNF

TABLE R06	comment
Keys	{ id }
Functional Dependencies:	
FD0601	id → {content, date, reports, edited, id_user, id_question, id_answer}
NORMAL FORM	BCNF

TABLE R07	notification
Keys	{ id }
Functional Dependencies:	
FD0701	id → {date, viewed, id_user, id_answer, id_comment, id_question_vote, id_answer_vote}
NORMAL FORM	BCNF

TABLE R08	follow_tag
Keys	{id_user, id_tag}
Functional Dependencies:	None
NORMAL FORM	BCNF

TABLE R09	follow_question
Keys	{id_user, id_question}
Functional Dependencies:	None
NORMAL FORM	BCNF

TABLE R10	question_vote
Keys	{ id }
Functional Dependencies:	
FD1001	id → {id_user, id_question,value}
NORMAL FORM	BCNF

TABLE R11	answer_vote
Keys	{ id }
Functional Dependencies:	
FD1101	id → {id_user, id_answer,value}
NORMAL FORM	BCNF

TABLE R12	image
Keys	{ id }
Functional Dependencies:	
FD1201	id → {imagePath, id_user, id_question}
NORMAL FORM	BCNF

TABLE R13	question_tag
Keys	{id_question, id_tag}
Functional Dependencies:	None

TABLE R13	question_tag
NORMAL FORM	BCNF

According to Boyce–Codd Normal Form (BCNF), for all non-trivial relations of type $A \rightarrow B$, it is a necessary condition to be a (super) key. Because all relations are in the BCNF, the relational schema is also in the BCNF.

A6: Indexes, triggers, transactions and database population

The goal of A6 is to show the database workload, indexes, triggers, and transactions. The complete database is also presented.

1. Database Workload

Relation reference	Relation Name	Order of magnitude	Estimated growth
R01	Users	thousands	units per day
R02	Report	hundreds	units per week
R03	Tag	dozens	units per month
R04	Question	tens thousands	dozens per day
R05	Answer	tens thousands	dozens per day
R06	Comment	thousands	units per day
R07	Notification	hundreds of thousands	hundreds per day
R08	FollowTag	tens thousands	dozens per day
R09	FollowQuestion	tens thousands	dozens per day
R10	QuestionVote	thousands	units per day
R11	AnswerVote	thousands	units per day
R12	Image	hundreds	units per week
R13	QuestionTag	tens thousands	dozens per day

2. Proposed Indices

2.1. Performance Indices

Index	IDX01
Relation	Users
Attribute	email
Type	B-tree
Cardinality	High
Clustering	No
Justification	The User table can grow significantly, and several operations frequently need to locate users by email, especially during login and authentication checks. Since the search is done by exact match, a B-tree index is the most appropriate. Additionally, the email is a unique key, so it's a good index.
SQL code	CREATE UNIQUE INDEX idx_user_email ON Users (email);

Index	IDX02
Relation	Question
Attribute	votes, reports

Index	IDX02
Type	B-tree
Cardinality	High
Clustering	No
Justification	Questions with the most votes and reports are often sorted according to these metrics to be displayed to users. Sorting by votes, in particular, will be common in scenarios where popular questions are prioritized. The choice of a B-tree index is due to the fact that read operations are frequent and require sorting, something that this type of index does efficiently. The cardinality is high because there is a large number of questions with different vote and report counts.
SQL code	CREATE INDEX idx_question_votes_reports ON Question (votes DESC, reports DESC);

Index	IDX03
Relation	Answer
Attribute	id_question, votes
Type	B-tree
Cardinality	Medium
Clustering	Yes
Justification	This index will be used for frequent queries that seek answers associated with a specific question, sorted by the number of votes. The use of clustering in this case makes sense because answers are always retrieved based on the question, and sorting by votes is common. The cardinality is medium, as a question can have multiple answers, but there is a limited number of questions.
SQL code	CREATE INDEX idx_answer_question ON Answer (id_question, votes DESC); CLUSTER Answer USING idx_answer_question;

2.2. Full-text Search Indices

Index	IDX11
Relation	Question
Attribute	title, content
Type	GIN
Clustering	No
Justification	The index was created to allow full-text search on questions, based on the title and content. This will help users find specific questions based on keywords in the titles or descriptions. The GIN index is the recommended index for large volumes of textual data and for full-text search queries.
SQL code	

```
-- Add column to the Question table to store ts_vectors.
ALTER TABLE Question
ADD COLUMN tsvectors TSVECTOR;

-- Create a function to automatically update the ts_vectors.
CREATE FUNCTION question_search_update() RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    NEW.tsvectors = (
      setweight(to_tsvector('english', NEW.title), 'A') ||
      setweight(to_tsvector('english', NEW.content), 'B')
    );
  END IF;
  IF TG_OP = 'UPDATE' THEN
    IF (NEW.title <> OLD.title OR NEW.content <> OLD.content) THEN
      NEW.tsvectors = (
        setweight(to_tsvector('english', NEW.title), 'A') ||
        setweight(to_tsvector('english', NEW.content), 'B')
      );
    END IF;
  END IF;
END;
```

```
);
END IF;
END IF;
RETURN NEW;
END $$
LANGUAGE plpgsql;

-- Create a trigger before inserting or updating in the Question table.
CREATE TRIGGER question_search_update
  BEFORE INSERT OR UPDATE ON Question
  FOR EACH ROW
  EXECUTE PROCEDURE question_search_update();

-- Create the GIN index for the ts_vectors.
CREATE INDEX question_search_idx ON Question USING GIN (tsvectors);
```

Index	IDX12
Relation	Tag
Attribute	name
Type	GIN
Clustering	No
Justification	The index allows users to search for tags associated with questions, providing a fast and efficient way to find content related to keywords. Since tags are an important part of organizing questions, searching by name will enhance navigation and the discovery of relevant content.
SQL code	

```
-- Add column to the Tag table to store ts_vectors.
ALTER TABLE Tag
ADD COLUMN tsvectors TSVECTOR;

-- Create a function to automatically update the ts_vectors.
CREATE FUNCTION tag_search_update() RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    NEW.tsvectors = (
      setweight(to_tsvector('english', NEW.name), 'A') ||
      setweight(to_tsvector('english', NEW.creation_date::text), 'B')
    );
  END IF;
  IF TG_OP = 'UPDATE' THEN
    IF (NEW.name <> OLD.name OR NEW.creation_date <> OLD.creation_date) THEN
      NEW.tsvectors = (
        setweight(to_tsvector('english', NEW.name), 'A') ||
        setweight(to_tsvector('english', NEW.creation_date::text), 'B')
      );
    END IF;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

-- Create a trigger before inserting or updating in the Tag table.
CREATE TRIGGER tag_search_update
  BEFORE INSERT OR UPDATE ON tag
  FOR EACH ROW
  EXECUTE PROCEDURE tag_search_update();

-- Create the GIN index for the ts_vectors.
CREATE INDEX search_idx ON Tag USING GIN (tsvectors);
```


3. Triggers

Trigger	TRIGGER01
Description	Clear indication of edits in comments and answers
SQL code	

```
CREATE FUNCTION mark_answer_comment_as_edited() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF (NEW.content <> OLD.content) THEN
            NEW.edited = TRUE;
            NEW.date = NOW();
        END IF;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER answer_edited
BEFORE UPDATE ON Answer
FOR EACH ROW
EXECUTE PROCEDURE mark_answer_comment_as_edited();

CREATE TRIGGER comment_edited
BEFORE UPDATE ON Comment
FOR EACH ROW
EXECUTE PROCEDURE mark_answer_comment_as_edited();
```

Trigger	TRIGGER02
Description	Users cannot interact with their own posts
SQL code	

```
CREATE FUNCTION prevent_self_interaction() RETURNS TRIGGER AS $$
DECLARE
    post_user_id INTEGER;
BEGIN
    IF TG_TABLE_NAME = 'Comment' THEN
        SELECT id_user INTO post_user_id FROM Question WHERE id = NEW.id_question;
    ELSIF TG_TABLE_NAME = 'Answer' THEN
        SELECT id_user INTO post_user_id FROM Question WHERE id = NEW.id_question;
    END IF;

    IF NEW.id_user = post_user_id THEN
        RAISE EXCEPTION 'Users cannot review or comment their own posts';
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER prevent_self_comment
BEFORE INSERT ON Comment
FOR EACH ROW
EXECUTE PROCEDURE prevent_self_interaction();

CREATE TRIGGER prevent_self_answer
BEFORE INSERT ON Answer
FOR EACH ROW
EXECUTE PROCEDURE prevent_self_interaction();
```

Trigger	TRIGGER03
Description	Notification to the question author when receiving an answer
SQL code	

```
CREATE FUNCTION notify_question_author() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Notification (date, viewed, id_user, id_answer, id_comment, id_question_vote, id_answer_vote)
    VALUES (
        NOW(),
        FALSE,
        (SELECT id_user FROM Question WHERE id = NEW.id_question),
        NEW.id,
        NULL,
        NULL,
        NULL
    );
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER answer_notification
AFTER INSERT ON Answer
FOR EACH ROW
EXECUTE FUNCTION notify_question_author();
```

Trigger	TRIGGER04
Description	Notification to the answer author when receiving a new comment
SQL code	

```
CREATE FUNCTION notify_answer_author() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Notification (date, viewed, id_user, id_answer, id_comment, id_question_vote, id_answer_vote)
    VALUES (
        NOW(),
        FALSE,
        (SELECT id_user FROM Answer WHERE id = NEW.id_answer),
        NULL,
        NEW.id,
        NULL,
        NULL
    );
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER comment_notification
AFTER INSERT ON Comment
FOR EACH ROW
EXECUTE FUNCTION notify_answer_author();
```

Trigger	TRIGGER05
Description	Update the number of reports when a report is made
SQL code	

```
CREATE FUNCTION update_reports() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_question IS NOT NULL THEN
        UPDATE Question
```

```
        SET reports = reports + 1
        WHERE id = NEW.id_question;
    ELSIF NEW.id_answer IS NOT NULL THEN
        UPDATE Answer
        SET reports = reports + 1
        WHERE id = NEW.id_answer;
    ELSIF NEW.id_comment IS NOT NULL THEN
        UPDATE Comment
        SET reports = reports + 1
        WHERE id = NEW.id_comment;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;
```

```
CREATE TRIGGER update_reports_count
AFTER INSERT ON Report
FOR EACH ROW
EXECUTE PROCEDURE update_reports();
```

Trigger	TRIGGER06
Description	Update the user score based on votes and actions
SQL code	

```
CREATE FUNCTION update_user_score() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.votes > OLD.votes THEN
        IF NEW.votes - OLD.votes > 0 THEN
            UPDATE Users SET score = score + 10 WHERE id = NEW.id_user;  -- Positive vote
        ELSE
            UPDATE Users SET score = score - 2 WHERE id = NEW.id_user;   -- Negative vote
        END IF;
    ELSIF NEW.accepted = TRUE THEN
        UPDATE Users SET score = score + 15 WHERE id = NEW.id_user;  -- Accept answer
        UPDATE Users SET score = score + 2 WHERE id = (SELECT id_user FROM Question WHERE id = NEW.id_question);  -- Use
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE Users SET score = score - 100 WHERE id = OLD.id_user;  -- Penalty
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER update_user_score_on_vote
AFTER INSERT OR UPDATE ON QuestionVote
FOR EACH ROW
EXECUTE PROCEDURE update_user_score();

CREATE TRIGGER update_user_score_on_answer_vote
AFTER INSERT OR UPDATE ON AnswerVote
FOR EACH ROW
EXECUTE PROCEDURE update_user_score();

CREATE TRIGGER update_user_score_on_acceptance
AFTER UPDATE ON Answer
FOR EACH ROW
WHEN (NEW.accepted = TRUE)
EXECUTE PROCEDURE update_user_score();
```

Trigger	TRIGGER07
Description	An answer can only be marked as accepted by the question author

Trigger	TRIGGER07
SQL code	

```
CREATE OR REPLACE FUNCTION check_author_accept_answer()
RETURNS TRIGGER AS $$
BEGIN

    IF NEW.accepted = TRUE THEN
        PERFORM 1
        FROM Question
        WHERE id = NEW.id_question AND id_user = NEW.id_user;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'Only the author of the question can accept an answer';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_author_accept_answer
BEFORE UPDATE OF accepted ON Answer
FOR EACH ROW
WHEN (OLD.accepted = FALSE AND NEW.accepted = TRUE)
EXECUTE FUNCTION check_author_accept_answer();
```

Trigger	TRIGGER08
Description	A user cannot report the same answer/comment/question/user more than once
SQL code	

```
CREATE OR REPLACE FUNCTION check_duplicate_report()
RETURNS TRIGGER AS $$
BEGIN

    IF EXISTS (
        SELECT 1
        FROM Report
        WHERE id_user = NEW.id_user
        AND (
            (id_question IS NOT NULL AND id_question = NEW.id_question) OR
            (id_answer IS NOT NULL AND id_answer = NEW.id_answer) OR
            (id_comment IS NOT NULL AND id_comment = NEW.id_comment)
        )
    ) THEN
        RAISE EXCEPTION 'User cannot report the same entity more than once';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_duplicate_report
BEFORE INSERT ON Report
FOR EACH ROW
EXECUTE FUNCTION check_duplicate_report();
```

Trigger	TRIGGER09
Description	A report must be associated with only one entity

Trigger	TRIGGER09
SQL code	

```
CREATE OR REPLACE FUNCTION enforce_single_report_association()
RETURNS TRIGGER AS $$
BEGIN

    IF (NEW.id_answer IS NOT NULL)::int + (NEW.id_comment IS NOT NULL)::int + (NEW.id_question IS NOT NULL)::int <> 1 THEN
        RAISE EXCEPTION 'A report must be associated with only one of: answer, comment, or question';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_enforce_single_report_association
BEFORE INSERT OR UPDATE ON Report
FOR EACH ROW
EXECUTE FUNCTION enforce_single_report_association();
```

Trigger	TRIGGER10
Description	A question can be associated with a maximum of 5 tags
SQL code	

```
CREATE OR REPLACE FUNCTION enforce_question_tag_limit()
RETURNS TRIGGER AS $$
BEGIN

    IF (SELECT COUNT(*) FROM QuestionTag WHERE id_question = NEW.id_question) >= 5 THEN
        RAISE EXCEPTION 'A question can be associated with a maximum of 5 tags';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_enforce_question_tag_limit
BEFORE INSERT ON QuestionTag
FOR EACH ROW
EXECUTE FUNCTION enforce_question_tag_limit();
```

Trigger	TRIGGER11
Description	Clear indication of edits in questions
SQL code	

```
CREATE FUNCTION mark_question_as_edited() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF (NEW.content <> OLD.content OR NEW.title <> OLD.title) THEN
            NEW.edited = TRUE;
            NEW.date = NOW();
        END IF;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER question_edited
```

```
BEFORE UPDATE ON Question
FOR EACH ROW
EXECUTE PROCEDURE mark_question_as_edited();
```

4. Transactions

SQL Reference	Delete Account
Justification	Allows users to permanently delete their accounts and personal data.
Isolation level	SERIALIZABLE
Complete SQL Code	

```
BEGIN TRANSACTION;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;


UPDATE comment
SET id_user = NULL,
WHERE id_user = $1;

UPDATE answer
SET id_user = NULL,
WHERE id_user = $1;

UPDATE question
SET id_user = NULL
WHERE id_user = $1;

UPDATE report
SET id_user = NULL
WHERE id_user = $1;

UPDATE notification
SET id_user = NULL
WHERE id_user = $1;

UPDATE followtag
SET id_user = NULL
WHERE id_user = $1;

UPDATE followquestion
SET id_user = NULL
WHERE id_user = $1;

UPDATE questionvote
SET id_user = NULL
WHERE id_user = $1;

UPDATE answervote
SET id_user = NULL
WHERE id_user = $1;

DELETE FROM users WHERE id = $1;

END TRANSACTION;
```

SQL Reference	Delete Question
Justification	Allows users to remove questions that are no longer relevant

SQL Reference	Delete Question
Isolation level	SERIALIZABLE
Complete SQL Code	

```
BEGIN TRANSACTION;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

DELETE FROM questionvote WHERE id_question = $1;
DELETE FROM followquestion WHERE id_question = $1;
DELETE FROM report WHERE id_question = $1;
DELETE FROM comment WHERE id_question = $1;
DELETE FROM answer WHERE id_question = $1;
DELETE FROM question WHERE id = $1;

END TRANSACTION;
```

SQL Reference	Delete Answer
Justification	Allows users to remove unwanted answers.
Isolation level	SERIALIZABLE
Complete SQL Code	

```
BEGIN TRANSACTION;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

DELETE FROM answervote WHERE id_answer = $1;
DELETE FROM report WHERE id_answer = $1;
DELETE FROM comment WHERE id_answer = $1;
DELETE FROM answer WHERE id = $1;

END TRANSACTION;
```

Annex A. SQL Code

A.1. Database schema

```
DROP TABLE IF EXISTS Users CASCADE;
DROP TABLE IF EXISTS Badge CASCADE;
DROP TABLE IF EXISTS Report CASCADE;
DROP TABLE IF EXISTS Tag CASCADE;
DROP TABLE IF EXISTS Question CASCADE;
DROP TABLE IF EXISTS Answer CASCADE;
DROP TABLE IF EXISTS Comment CASCADE;
DROP TABLE IF EXISTS Notification CASCADE;
DROP TABLE IF EXISTS FollowTag CASCADE;
DROP TABLE IF EXISTS FollowQuestion CASCADE;
DROP TABLE IF EXISTS User_Badge CASCADE;
DROP TABLE IF EXISTS QuestionVote CASCADE;
DROP TABLE IF EXISTS AnswerVote CASCADE;
DROP TABLE IF EXISTS Image CASCADE;
DROP TABLE IF EXISTS QuestionTag CASCADE;
DROP TRIGGER IF EXISTS question_search_update ON Question;
DROP TRIGGER IF EXISTS tag_search_update ON Tag;
DROP TRIGGER IF EXISTS question_edited ON Question;
DROP TRIGGER IF EXISTS answer_edited ON Answer;
DROP TRIGGER IF EXISTS prevent_self_comment ON Comment;
DROP TRIGGER IF EXISTS prevent_self_answer ON Answer;
DROP TRIGGER IF EXISTS answer_notification ON Answer;
```

```
DROP TRIGGER IF EXISTS comment_notification ON Comment;
DROP TRIGGER IF EXISTS update_reports_count ON Report;
DROP TRIGGER IF EXISTS update_user_score_on_vote ON QuestionVote;
DROP TRIGGER IF EXISTS update_user_score_on_answer_vote ON AnswerVote;
DROP TRIGGER IF EXISTS update_user_score_on_acceptance ON Answer;
DROP TRIGGER IF EXISTS trigger_check_author_accept_answer ON Answer;
DROP TRIGGER IF EXISTS trigger_check_duplicate_report ON Report;
DROP TRIGGER IF EXISTS trigger_enforce_single_report_association ON Report;
DROP TRIGGER IF EXISTS trigger_enforce_question_tag_limit ON QuestionTag;
DROP FUNCTION IF EXISTS question_search_update;
DROP FUNCTION IF EXISTS tag_search_update;
DROP FUNCTION IF EXISTS mark_as_edited;
DROP FUNCTION IF EXISTS prevent_self_interaction;
DROP FUNCTION IF EXISTS notify_question_author;
DROP FUNCTION IF EXISTS notify_answer_author;
DROP FUNCTION IF EXISTS update_reports;
DROP FUNCTION IF EXISTS update_user_score;
DROP FUNCTION IF EXISTS check_author_accept_answer;
DROP FUNCTION IF EXISTS check_duplicate_report;
DROP FUNCTION IF EXISTS enforce_single_report_association;
DROP FUNCTION IF EXISTS enforce_question_tag_limit;
DROP FUNCTION IF EXISTS mark_answer_comment_as_edited;
DROP FUNCTION IF EXISTS mark_question_as_edited;

CREATE TABLE IF NOT EXISTS Users (
    id SERIAL PRIMARY KEY,
    admin BOOLEAN DEFAULT FALSE NOT NULL,
    username VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    bio TEXT,
    birthdate DATE CONSTRAINT valid_birthdate CHECK (birthdate <= CURRENT_DATE),
    password VARCHAR(255) NOT NULL,
    signUpDate DATE CONSTRAINT valid_signUpDate CHECK (signUpDate <= CURRENT_DATE),
    ban BOOLEAN DEFAULT FALSE NOT NULL,
    score INTEGER DEFAULT 0 NOT NULL
);

CREATE TABLE IF NOT EXISTS Tag (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) UNIQUE NOT NULL,
    creation_date DATE DEFAULT CURRENT_DATE NOT NULL
);

CREATE TABLE IF NOT EXISTS Question (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    date DATE DEFAULT CURRENT_DATE NOT NULL,
    reports INTEGER DEFAULT 0 NOT NULL CONSTRAINT valid_reports CHECK (reports >= 0),
    votes INTEGER DEFAULT 0 NOT NULL,
    edited BOOLEAN DEFAULT FALSE NOT NULL,
    id_user INTEGER REFERENCES Users(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Answer (
    id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    date DATE DEFAULT CURRENT_DATE NOT NULL,
    reports INTEGER DEFAULT 0 NOT NULL CONSTRAINT valid_reports CHECK (reports >= 0),
    votes INTEGER DEFAULT 0 NOT NULL,
    accepted BOOLEAN DEFAULT FALSE NOT NULL,
    edited BOOLEAN DEFAULT FALSE NOT NULL,
    id_user INTEGER REFERENCES Users(id) ON DELETE SET NULL,
    id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE
);
```



```
CREATE TABLE IF NOT EXISTS Comment (  
  id SERIAL PRIMARY KEY,  
  content TEXT NOT NULL,  
  date DATE DEFAULT CURRENT_DATE NOT NULL,  
  reports INTEGER DEFAULT 0 NOT NULL CONSTRAINT valid_reports CHECK (reports >= 0),  
  edited BOOLEAN DEFAULT FALSE NOT NULL,  
  id_user INTEGER REFERENCES Users(id) ON DELETE SET NULL,  
  id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,  
  id_answer INTEGER REFERENCES Answer(id) ON DELETE CASCADE,  
  CONSTRAINT valid_comment CHECK ((id_question IS NOT NULL AND id_answer IS NULL) OR (id_question IS NULL AND id_answer IS NOT NULL))  
);  
  
CREATE TABLE IF NOT EXISTS QuestionVote (  
  id SERIAL PRIMARY KEY,  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,  
  value INTEGER CONSTRAINT valid_vote CHECK (value = 1 OR value = -1)  
);  
  
CREATE TABLE IF NOT EXISTS AnswerVote (  
  id SERIAL PRIMARY KEY,  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_answer INTEGER REFERENCES Answer(id) ON DELETE CASCADE,  
  value INTEGER CONSTRAINT valid_vote CHECK (value = 1 OR value = -1)  
);  
  
CREATE TABLE IF NOT EXISTS Notification (  
  id SERIAL PRIMARY KEY,  
  date TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  viewed BOOLEAN DEFAULT FALSE NOT NULL,  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_answer INTEGER REFERENCES Answer(id) ON DELETE CASCADE,  
  id_comment INTEGER REFERENCES Comment(id) ON DELETE CASCADE,  
  id_question_vote INTEGER REFERENCES QuestionVote(id) ON DELETE CASCADE,  
  id_answer_vote INTEGER REFERENCES AnswerVote(id) ON DELETE CASCADE,  
  CONSTRAINT valid_notification CHECK ((id_answer IS NOT NULL AND id_comment IS NULL AND id_question_vote IS NULL AND id_answer_vote IS NULL)  
  OR (id_answer IS NULL AND id_comment IS NOT NULL AND id_question_vote IS NULL AND id_answer_vote IS NULL)  
  OR (id_answer IS NULL AND id_comment IS NULL AND id_question_vote IS NOT NULL AND id_answer_vote IS NULL)  
  OR (id_answer IS NULL AND id_comment IS NULL AND id_question_vote IS NULL AND id_answer_vote IS NOT NULL))  
);  
  
CREATE TABLE IF NOT EXISTS Report (  
  id SERIAL PRIMARY KEY,  
  content TEXT NOT NULL,  
  date DATE DEFAULT CURRENT_DATE NOT NULL,  
  viewed BOOLEAN DEFAULT FALSE NOT NULL,  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,  
  id_answer INTEGER REFERENCES Answer(id) ON DELETE CASCADE,  
  id_comment INTEGER REFERENCES Comment(id) ON DELETE CASCADE,  
  CONSTRAINT valid_report CHECK ((id_question IS NOT NULL AND id_answer IS NULL AND id_comment IS NULL) OR (id_question IS NULL AND id_answer IS NOT NULL AND id_comment IS NULL))  
);  
  
CREATE TABLE IF NOT EXISTS FollowTag (  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_tag INTEGER REFERENCES Tag(id) ON DELETE CASCADE,  
  PRIMARY KEY (id_user, id_tag)  
);  
  
CREATE TABLE IF NOT EXISTS FollowQuestion (  
  id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,  
  id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,  
  PRIMARY KEY (id_user, id_question)  
);
```

```

CREATE TABLE IF NOT EXISTS Image (
    id SERIAL PRIMARY KEY,
    imagePath VARCHAR(255) NOT NULL,
    id_user INTEGER REFERENCES Users(id) ON DELETE CASCADE,
    id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,
    CONSTRAINT valid_image CHECK ((id_question IS NOT NULL AND id_user IS NULL) OR (id_question IS NULL AND id_user IS NOT NULL));
);

CREATE TABLE IF NOT EXISTS QuestionTag (
    id_question INTEGER REFERENCES Question(id) ON DELETE CASCADE,
    id_tag INTEGER REFERENCES Tag(id) ON DELETE CASCADE,
    PRIMARY KEY (id_question, id_tag)
);

CREATE UNIQUE INDEX idx_user_email ON Users (email);

CREATE INDEX idx_question_votes_reports ON Question (votes DESC, reports DESC);

CREATE INDEX idx_answer_question ON Answer (id_question, votes DESC);CLUSTER Answer USING idx_answer_question;

-- Add column to the Question table to store ts_vectors.
ALTER TABLE Question
ADD COLUMN tsvectors TSVECTOR;

-- Create a function to automatically update the ts_vectors.
CREATE FUNCTION question_search_update() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.tsvectors = (
            setweight(to_tsvector('english', NEW.title), 'A') ||
            setweight(to_tsvector('english', NEW.content), 'B')
        );
    END IF;
    IF TG_OP = 'UPDATE' THEN
        IF (NEW.title <> OLD.title OR NEW.content <> OLD.content) THEN
            NEW.tsvectors = (
                setweight(to_tsvector('english', NEW.title), 'A') ||
                setweight(to_tsvector('english', NEW.content), 'B')
            );
        END IF;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

-- Create a trigger before inserting or updating in the Question table.
CREATE TRIGGER question_search_update
BEFORE INSERT OR UPDATE ON Question
FOR EACH ROW
EXECUTE PROCEDURE question_search_update();

-- Create the GIN index for the ts_vectors.
CREATE INDEX question_search_idx ON Question USING GIN (tsvectors);

ALTER TABLE Tag
ADD COLUMN tsvectors TSVECTOR;

-- Create a function to automatically update the ts_vectors.
CREATE FUNCTION tag_search_update() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.tsvectors = (
            setweight(to_tsvector('english', NEW.name), 'A') ||

```

```
        setweight(to_tsvector('english', NEW.creation_date::text), 'B')
    );
END IF;
IF TG_OP = 'UPDATE' THEN
    IF (NEW.name <> OLD.name OR NEW.creation_date <> OLD.creation_date) THEN
        NEW.tsvectors = (
            setweight(to_tsvector('english', NEW.name), 'A') ||
            setweight(to_tsvector('english', NEW.creation_date::text), 'B')
        );
    END IF;
END IF;
RETURN NEW;
END $$ LANGUAGE plpgsql;

-- Create a trigger before inserting or updating in the Tag table.
CREATE TRIGGER tag_search_update
BEFORE INSERT OR UPDATE ON tag
FOR EACH ROW
EXECUTE PROCEDURE tag_search_update();

-- Create the GIN index for the ts_vectors.
CREATE INDEX search_idx ON Tag USING GIN (tsvectors);

CREATE FUNCTION mark_question_as_edited() RETURNS TRIGGER AS $$
BEGIN
IF TG_OP = 'UPDATE' THEN
IF (NEW.content <> OLD.content OR NEW.title <> OLD.title) THEN
NEW.edited = TRUE;
NEW.date = NOW();
END IF;
END IF;
RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER question_edited
BEFORE UPDATE ON Question
FOR EACH ROW
EXECUTE PROCEDURE mark_question_as_edited();

CREATE FUNCTION mark_answer_comment_as_edited() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF (NEW.content <> OLD.content) THEN
            NEW.edited = TRUE;
            NEW.date = NOW();
        END IF;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER answer_edited
BEFORE UPDATE ON Answer
FOR EACH ROW
EXECUTE PROCEDURE mark_answer_comment_as_edited();

CREATE TRIGGER comment_edited
BEFORE UPDATE ON Comment
FOR EACH ROW
EXECUTE PROCEDURE mark_answer_comment_as_edited();

CREATE FUNCTION prevent_self_interaction() RETURNS TRIGGER AS $$
DECLARE
```

```
    post_id_user INTEGER;
BEGIN
IF TG_TABLE_NAME = 'Comment' THEN
    SELECT id_user INTO post_id_user FROM Question WHERE id = NEW.id_question;
ELSIF TG_TABLE_NAME = 'Answer' THEN
    SELECT id_user INTO post_id_user FROM Question WHERE id = NEW.id_question;
END IF;

IF NEW.id_user = post_id_user THEN
RAISE EXCEPTION 'Users cannot review or comment their own posts';
END IF;
RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER prevent_self_comment
BEFORE INSERT ON Comment
FOR EACH ROW
EXECUTE PROCEDURE prevent_self_interaction();

CREATE TRIGGER prevent_self_answer
BEFORE INSERT ON Answer
FOR EACH ROW
EXECUTE PROCEDURE prevent_self_interaction();

CREATE FUNCTION notify_question_author() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Notification (date, viewed, id_user, id_answer, id_comment, id_question_vote, id_answer_vote)
VALUES (
    NOW(),
    FALSE,
    (SELECT id_user FROM Question WHERE id = NEW.id_question),
    NEW.id,
    NULL,
    NULL,
    NULL
);
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER answer_notification
AFTER INSERT ON Answer
FOR EACH ROW
EXECUTE FUNCTION notify_question_author();

CREATE FUNCTION notify_answer_author() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Notification (date, viewed, id_user, id_answer, id_comment, id_question_vote, id_answer_vote)
VALUES (
    NOW(),
    FALSE,
    (SELECT id_user FROM Answer WHERE id = NEW.id_answer),
    NULL,
    NEW.id,
    NULL,
    NULL
);
    RETURN NEW;
END $$
LANGUAGE plpgsql;

CREATE TRIGGER comment_notification
AFTER INSERT ON Comment
FOR EACH ROW
```

```
EXECUTE FUNCTION notify_answer_author();
```

```
CREATE FUNCTION update_reports() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_question IS NOT NULL THEN
        UPDATE Question
        SET reports = reports + 1
        WHERE id = NEW.id_question;
    ELSIF NEW.id_answer IS NOT NULL THEN
        UPDATE Answer
        SET reports = reports + 1
        WHERE id = NEW.id_answer;
    ELSIF NEW.id_comment IS NOT NULL THEN
        UPDATE Comment
        SET reports = reports + 1
        WHERE id = NEW.id_comment;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;
```

```
CREATE TRIGGER update_reports_count
AFTER INSERT ON Report
FOR EACH ROW
EXECUTE PROCEDURE update_reports();
```

```
CREATE FUNCTION update_user_score() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.votes > OLD.votes THEN
        IF NEW.votes - OLD.votes > 0 THEN
            UPDATE Users SET score = score + 10 WHERE id = NEW.id_user;
        ELSE
            UPDATE Users SET score = score - 2 WHERE id = NEW.id_user;
        END IF;
    ELSIF NEW.accepted = TRUE THEN
        UPDATE Users SET score = score + 15 WHERE id = NEW.id_user;
        UPDATE Users SET score = score + 2 WHERE id = (SELECT id_user FROM Question WHERE id = NEW.id_question);
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE Users SET score = score - 100 WHERE id = OLD.id_user;
    END IF;
    RETURN NEW;
END $$
LANGUAGE plpgsql;
```

```
CREATE TRIGGER update_user_score_on_vote
AFTER INSERT OR UPDATE ON QuestionVote
FOR EACH ROW
EXECUTE PROCEDURE update_user_score();
```

```
CREATE TRIGGER update_user_score_on_answer_vote
AFTER INSERT OR UPDATE ON AnswerVote
FOR EACH ROW
EXECUTE PROCEDURE update_user_score();
```

```
CREATE TRIGGER update_user_score_on_acceptance
AFTER UPDATE ON Answer
FOR EACH ROW
WHEN (NEW.accepted = TRUE)
EXECUTE PROCEDURE update_user_score();
```

```
CREATE OR REPLACE FUNCTION check_author_accept_answer()
RETURNS TRIGGER AS $$
BEGIN
```

```
IF NEW.accepted = TRUE THEN
    PERFORM 1
    FROM Question
    WHERE id = NEW.id_question AND id_user = NEW.id_user;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Only the author of the question can accept an answer';
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_author_accept_answer
BEFORE UPDATE OF accepted ON Answer
FOR EACH ROW
WHEN (OLD.accepted = FALSE AND NEW.accepted = TRUE)
EXECUTE FUNCTION check_author_accept_answer();

CREATE OR REPLACE FUNCTION check_duplicate_report()
RETURNS TRIGGER AS $$
BEGIN

    IF EXISTS (
        SELECT 1
        FROM Report
        WHERE id_user = NEW.id_user
        AND (
            (id_question IS NOT NULL AND id_question = NEW.id_question) OR
            (id_answer IS NOT NULL AND id_answer = NEW.id_answer) OR
            (id_comment IS NOT NULL AND id_comment = NEW.id_comment)
        )
    ) THEN
        RAISE EXCEPTION 'User cannot report the same entity more than once';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_duplicate_report
BEFORE INSERT ON Report
FOR EACH ROW
EXECUTE FUNCTION check_duplicate_report();

CREATE OR REPLACE FUNCTION enforce_single_report_association()
RETURNS TRIGGER AS $$
BEGIN

    IF (NEW.id_answer IS NOT NULL)::int + (NEW.id_comment IS NOT NULL)::int + (NEW.id_question IS NOT NULL)::int <> 1 THEN
        RAISE EXCEPTION 'A report must be associated with only one of: answer, comment, or question';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_enforce_single_report_association
BEFORE INSERT OR UPDATE ON Report
FOR EACH ROW
EXECUTE FUNCTION enforce_single_report_association();

CREATE OR REPLACE FUNCTION enforce_question_tag_limit()
```



```
RETURNS TRIGGER AS $$
BEGIN

    IF (SELECT COUNT(*) FROM QuestionTag WHERE id_question = NEW.id_question) >= 5 THEN
        RAISE EXCEPTION 'A question can be associated with a maximum of 5 tags';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_enforce_question_tag_limit
BEFORE INSERT ON QuestionTag
FOR EACH ROW
EXECUTE FUNCTION enforce_question_tag_limit();
```

A.2. Database population

```
INSERT INTO Users (admin, username, name, email, bio, birthdate, password, signUpDate, ban, score) VALUES
(FALSE, 'user_with_questions', 'User One', 'user_with_questions@example.com', 'Bio of user one', '1990-01-01', 'password1', '2023-01-01', FALSE, 0),
(FALSE, 'user_with_answers', 'User Two', 'user_with_answers@example.com', 'Bio of user two', '1991-02-02', 'password2', '2023-02-02', FALSE, 0),
(FALSE, 'user_with_both', 'User Three', 'user_with_both@example.com', 'Bio of user three', '1992-03-03', 'password3', '2023-03-03', FALSE, 0),
(FALSE, 'user_with_notifications', 'User Four', 'user_with_notifications@example.com', 'Bio of user four', '1993-04-04', 'password4', '2023-04-04', FALSE, 0),
(FALSE, 'user_follow_tags', 'User Five', 'user_follow_tags@example.com', 'Bio of user five', '1994-05-05', 'password5', '2023-05-05', FALSE, 0),
(FALSE, 'user_with_reports', 'User Six', 'user_with_reports@example.com', 'Bio of user six', '1995-06-06', 'password6', '2023-06-06', FALSE, 0),
(FALSE, 'banned_user', 'User Seven', 'banned_user@example.com', 'Bio of user seven', '1996-07-07', 'password7', '2023-07-07', TRUE, 0),
(TRUE, 'admin1', 'Admin One', 'admin1@example.com', 'Bio of admin one', '1985-01-01', 'adminpassword', '2023-01-01', FALSE, 100);

INSERT INTO Tag (name) VALUES
('tag1'),
('tag2'),
('tag3');

INSERT INTO Question (title, content, date, reports, votes, edited, id_user) VALUES
('Question 1', 'Content of question 1', '2023-01-01', 0, 5, FALSE, 1), -- Question by user_with_questions
('Question 2', 'Content of question 2', '2023-02-02', 0, 3, FALSE, 3), -- Question by user_with_both
('Edited Question', 'Content of edited question', '2023-03-01', 0, 10, TRUE, 1), -- Edited question by user_with_questions
('Question by banned_user', 'Content of question by banned_user', '2023-04-01', 3, -2, FALSE, 7); -- Question by banned_user
```

Revision history

Changes made to the first submission:

- 1. Item 1
- 2. ..

GROUP24141, 07/11/2024

- Diogo Ramos, [up202207954@up.pt](#)
- Daniel Silva, [up201909935@up.pt](#) (Editor)
- Rafael Costa, [up202205013@up.pt](#)
- Tiago Pires, [up202208910@up.pt](#)